

Documentação Completa: Tensor Mage Arena (Jogo-MAGO-RL)

1. Introdução e Visão Geral

O projeto **Tensor Mage Arena** consiste em um ambiente de Aprendizado por Reforço (Reinforcement Learning - RL) desenhado para treinar um agente autônomo através de Self-Play contínuo. A arquitetura é construída sobre **JAX** e **Flax**, permitindo que a amostragem do ambiente e as atualizações de gradiente sejam massivamente paralelizadas em tensores, evitando gargalos de CPU e alocação de memória gráfica tradicional.

O objetivo do projeto é que o agente aprenda heurísticas avançadas de combate — como navegação labiríntica, desvio de projéteis (dodging), perseguição, e otimização de tempo —, tudo através de sinais densos de recompensa (*Reward Shaping*) e um sistema avançado de progressão suave (*Continuous Curriculum Learning*).

2. A Engine de Simulação (State Space e Action Space)

A classe `GameEngine` atua como o **Markov Decision Process (MDP)**. Diferente de jogos PyGame comuns, ela foi escrita puramente em operações vetoriais de Numpy para rodar em modo `headless` durante o treinamento, garantindo amostragens ultra-rápidas.

2.1. Espaço de Observações (State Space) $\mathcal{S}$

A rede neural não recebe uma "foto" estática, mas uma fusão de dados espaciais e escalares ao longo do tempo (Frame Stacking) para inferir

velocidades e direções (resolvendo o problema de estado oculto em MDPs). O estado S_t no tempo t é definido como:

- **Spatial Observation (Visão Matricial):** Um tensor de formato $(64, 64, 40)$. São 8 frames empilhados temporalmente, onde cada frame possui 5 canais de informação matricial (Paredes, Drops, Posição do Agente, Posição do Inimigo, Densidade de Projéteis Ativos).
- **Scalar Observation (Sensores Numéricos):** Um tensor $(8, 15)$. Contém o Histórico HP, cooldowns, timers, e o vetor unitário de mira ideal supervisionada $\vec{v}_{aim}$.

2.2. Espaço de Ações (Action Space) $\mathcal{A}$

O agente opera num domínio multi-modal (Múltiplas cabeças de saída): -

Movimentação (Discreta): Distância de Manhattan em 5 direções [Norte, Sul, Leste, Oeste, Parado]. - **Mira (Contínua):** Vetor normalizado no espaço

L_2 , definido por $\vec{u} = \frac{(x, y)}{\sqrt{x^2 + y^2}} + \epsilon$. -

Intenção Tática (Discreta - Auxiliar): Predição de um de 6 estados heurísticos (FSM) para regularizar as features extraídas pela rede.

2.3. Pathfinding Inimigo (A* com Cost Map de Risco)

A simulação não depende apenas de RL; para gerar o currículo, um inimigo com IA Heurística usa o Algoritmo A*. O diferencial matemático aqui é a Injeção de Custo de Ameaça no grafo de busca.

Seja $C(x, y)$ o custo base de pisar num bloco (x, y) . $C(x, y) = 1.0$

(espaço livre) $C_{quina}(x, y) = 2.0$ $\text{(se adjacente a uma parede, previne travamento)}$

A projeção de um projétil inimigo no tempo k (para $k \in \{0, 1, 2, 3\}$ passos) altera dinamicamente o custo do bloco na matriz: $C(x + v_x k, y + v_y k)$

$\mathrel{+} = 100.0$ Isso transforma magicamente o A de um mero buscador de caminhos para um algoritmo de evasão de ameaças*, pois o custo letal faz a heurística desviar dos tiros no ar.

3. Reward Shaping (Matemática da Recompensa)

Em RL clássico, recompensas esparsas (como receber $+1$ ao matar ou -1 ao morrer) tornam o aprendizado impossível em cenários densos. O projeto injeta recompensas auxiliares densas a cada tick (frame).

A Recompensa total por tick R_t é a soma de funções de ganho e penalidade: $R_t = R_{\text{dano}} + R_{\text{drop}} - P_{\text{tempo}} - P_{\text{campismo}} - P_{\text{ameaça}}$

- 1. Penalidade de Tempo (P_{tempo}):** Escala linearmente entre 0.005 e 0.01 por tick com o progresso do currículo. Garante que o agente minimize a duração do episódio $\min \sum_{t=0}^T t$.
- 2. Penalidade de Campismo (P_{campismo}):** Seja $\vec{p}_t$ a posição do agente no tempo t . Se $|\vec{p}_t - \vec{p}_{t-20}|_2 < 2.0$, aplica-se uma punição. Isso destroça o "Mode Collapse" onde o agente descobre que ficar parado é o modo mais seguro de sobreviver.
- 3. Punição de Alinhamento Balístico ($P_{\text{ameaça}}$):** Penaliza o agente severamente se ele permanecer na rota de colisão de um projétil inimigo. Seja $\vec{d}$ o vetor do projétil para o agente e $\vec{v}_{\text{proj}}$ a velocidade do tiro: $\text{Alinhamento} = \frac{|\vec{v}_{\text{proj}} \cdot \vec{d}|}{|\vec{v}_{\text{proj}}| |\vec{d}|}$. Se $\text{Alinhamento} > 0.85$, uma punição proporcional à proximidade é extraída: $P_{\text{ameaça}} \propto (15.0 - |\vec{d}|)$.

4. O Cérebro Neural: Arquitetura Actor-Critic e Episodic Memory

O modelo neural processa as matrizes e sensores para decidir ações (Actor) e prever o valor do estado (Critic).

4.1. Backbone Convolutacional e ResNet

A entrada visual passa por três blocos de Convolução bidimensional. No Nível de Complexidade 2, **Blocos ResNet** (Residual Networks) são ativados. As conexões residuais previnem o desvanecimento do gradiente (*Vanishing*

Gradient): $H(x) = \text{ReLU}(F(x, \{W_i\}) + x)$ Onde x é a entrada da camada e $F(x)$ representa a transformação matricial dos filtros convolucionais. Isso permite que a rede "memorize" a identidade do mapa caso os novos filtros degradem a feature visual original.

4.2. Memória Episódica (Self-Attention Layer)

Ao invés de uma LSTM clássica, a temporalidade (8 frames) é tratada por um mecanismo de atenção (Transformer). Para os embeddings $Z = [z_1, z_2, \dots, z_8]$, projetamos Matrizes de Query (Q), Key (K) e Value (V): $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$ A vantagem do Self-Attention sobre LSTMs é que o último estado oculto z_8 tem acesso " $O(1)$ " imediato e irrestrito ao que aconteceu em z_1 , aprendendo a correlacionar a trajetória balística de projéteis com precisão absurda sem a degradação temporal das células recorrentes.

4.3. Action Masking Algébrico (Masked Softmax)

A rede sempre gera *logits* brutos $L \in \mathbb{R}^5$ para o movimento. Para evitar que o agente tente andar contra a parede, as rotas inválidas recebem um logit negativo colossal: $L_{\text{safe}}^{(i)} = \begin{cases} L^{(i)} & \text{se } \text{mask}^{(i)} == \text{True} \\ -10^7 & \text{se } \text{mask}^{(i)} == \text{False} \end{cases}$ Quando submetido à função softmax (L_{safe}), a probabilidade das ações bloqueadas colapsa microscopicamente para 0.0 , garantindo que a amostragem categórica no JAX e os cálculos logarítmicos dos gradientes permaneçam estáveis, sem gerar temidos NaNs (Not a Number) por causa de $\log(0)$.

5. O Algoritmo de Treinamento: Proximal Policy Optimization (PPO)

O agente é treinado pelo poderoso PPO, algoritmo Policy Gradient de confiança da OpenAI (usado também no ChatGPT/InstructGPT).

5.1. Estimativa da Vantagem Generalizada (GAE)

O GAE calcula quão melhor foi a ação tomada em relação ao que o Critic esperava. O Erro de Diferença Temporal (TD Error) no frame t é: $\delta_t = R_t + \gamma V(S_{t+1}) - V(S_t)$ A Vantagem Generalizada $\hat{A}_t$ acumula os erros decaindo-os por um fator λ : $\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$

5.2. Função de Custo Clipada (Clipped Surrogate Loss)

Se a rede der um salto enorme nos pesos que arruine a política, ela vai desaprender tudo (Catastrophic Forgetting). O PPO impede isso cortando o tamanho máximo da atualização. Seja a razão de probabilidade $r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}$. A Perda da Política (Policy Loss) a ser minimizada é o recíproco do ganho máximo: $L^{\text{CLIP}}(\theta) = -\hat{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$ Com o hiperparâmetro de clipping $\epsilon = 0.2$, se a vantagem for positiva e a rede nova sugerir mais de 20% de aumento de probabilidade, o gradiente vira 0, freando a atualização excessiva.

6. Continuous Curriculum Learning (Interpolação Suave)

Ao invés de níveis "fase 1, fase 2", o treinamento é orquestrado por um escalar $\mathcal{C}_{\text{progress}} \in [0.0, 1.0]$. Se o agente obtiver lucro na Epoch, $\mathcal{C}_{\text{progress}}$ sobe $+0.001$.

A física de todos os atributos e da Engine (velocidade do projétil inimigo, HP base, Hitboxes) sofrem transição vetorial usando **Interpolação Linear (LERP)**: $\text{Atributo}(t) = \text{Min} + (\text{Max} - \text{Min}) \times \mathcal{C}_{\text{progress}}$

Exemplo em código (`engine.py`):

```
enemy_hp = self.lerp(30.0, 100.0, p) # De fraco a tanque gradualmente
```

Para habilidades discretas (ex: IA passa a usar *A ao invés de movimento aleatório*), usa-se *Threshold Masking**, onde a habilidade é desbloqueada abruptamente apenas quando o risco interpolado (LERP) já preparou as reações da rede neural, reduzindo choques de dificuldade e instabilidade nos gradientes.

7. Técnicas SOTA (State of the Art) para Regularização

7.1. Elastic Weight Consolidation (EWC)

Em currículos de longa duração, uma rede foca em desviar de projéteis (Dificuldade Alta) e pode "esquecer" de atirar (Dificuldade Baixa). O EWC protege sinapses críticas do passado. Uma âncora θ_{old} e a matriz de Fisher F_i medem a importância do parâmetro θ_i . A nova Loss sofre uma penalidade quadrática se a rede tentar mudar neurônios cruciais:
$$L_{total} = L_{PPO} + \frac{\lambda}{2} \sum_i F_i (\theta_i - \theta_{i, old})^2$$

7.2. Cosine Annealing with Warm Restarts (SGDR)

Ao invés de baixar a taxa de aprendizado (Learning Rate - LR) infinitamente até o agente estagnar em mínimos locais, a classe de otimização no `train.py` (via Optax) aplica resfriamentos cíclicos. O LR decai exponencialmente para perto de zero e então tem um salto agressivo para 3×10^{-4} (Warm Restart), permitindo que a geometria da descida de gradiente explore topologias novas no espaço paramétrico antes de sedimentar pesos localmente de novo.

7.3. Self-Play Assíncrono via Pool Histórico

Durante as partidas paralelizadas, em um percentual dos mundos oponente não é guiado pela Engine, mas por uma cópia congelada do próprio modelo de períodos passados (`historical_params_pool`). O JAX bloqueia retropropagação na rede oponente (`jax.lax.stop_gradient`) para salvar computação. Isso garante generalização infinita, pois à medida que o Agente principal fica escorregadio e afiado, seus oponentes acompanham o ritmo e as

táticas de forma autossustentável (A corrida armamentista do Reinforcement Learning).

8. Conclusão

O *Tensor Mage Arena* transcende um ambiente toy de RL. Através da fusão de processamento matricial (JAX/Flax), arquiteturas modulares temporais (Transformers e ResNets), regularização matemática (EWC e SGDR) e currículo ininterrupto, constrói-se um laboratório de hiper-complexidade onde capacidades cognitivas como Fuga, Perseguição, Pathfinding e Gestão de Risco emergem puramente a partir da maximização da equação de Bellman modificada por Reward Shaping tático.